Deep Learning for Credit Card Fraud Detection Threshold Optimization and Explainable AI Consensus

1st Sadman Bin Anwar Siam
Dept. Computer Science and Engineering
Brac University
Dhaka, Bangladesh
sadman.bin.anwar.siam@gmail.com

2nd Shabiba Binta Shahadat
Dept. Computer Science and Engineering
Brac University
Dhaka, Bangladesh
shabiba.binta.shahadat@g.bracu.ac.bd

3rd A Z M Bodruddoza Bijoy
Dept. Computer Science and Engineering
Brac University
Dhaka, Bangladesh
azm.bodruddoza.bijoy@g.bracu.ac.bd

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Credit card fraud detection has become a major challenge in modern digital financial systems due to the rapid growth of online transactions and the highly imbalanced nature of financial datasets. Traditional machine learning approaches often struggle to maintain high fraud detection performance while minimizing false positives. This paper proposes a deep learning-based credit card fraud detection framework integrating threshold optimization and Explainable Artificial Intelligence (XAI) consensus techniques. The proposed system combines Deep Neural Networks, Random Forest, and Logistic Regression models to improve prediction robustness and reliability. To address severe class imbalance, the Synthetic Minority Oversampling Technique (SMOTE) is applied during preprocessing. Furthermore, algorithmic threshold optimization is employed to maximize the F1-score and improve fraud sensitivity beyond the default classification threshold. SHAP and LIME are incorporated to provide interpretable explanations for model predictions and improve transparency in financial decision-making. Experimental evaluation on a real-world simulated credit card transaction dataset with approximately 1% fraudulent transactions demonstrates improved fraud detection capability with enhanced interpretability and reduced false positives. The proposed framework provides an adaptive and explainable solution suitable for modern financial fraud detection environments.**

***Index Terms*—Credit Card Fraud Detection, Deep Learning, Explainable AI, SHAP, LIME, Threshold Optimization, Machine Learning, Financial Security.**

I. Introduction

Among today's financial concerns, few weigh heavier than credit card fraud. With digital payments expanding steadily, deceptive tactics evolve in complexity and volume. Billions vanish each year from banking entities, yet numbers only tell part of the story - confidence fades when accounts are compromised, weakening global transaction networks slowly. Stability within finance hinges on reliability; breaches test that foundation repeatedly.Most challenges in spotting fraud stem from uneven distribution within transaction records. One category dominates so heavily - regular activity versus irregular - that standard systems struggle to adapt effectively under actual operating circumstances. Real-life datasets frequently show a hundredfold difference, tipping accuracy toward error when applying common analytical approaches.This work tackles existing issues through application of the Credit Card 2023 collection. Multiple approaches take shape: Logistic Regression appears first, followed by Random Forest, then a tailored Deep Neural Network enters the frame. Rather than rely on conventional metrics alone, an adjusted decision threshold emerges via Algorithmic Threshold Optimization. Such adjustment aims at peak F1-Score performance despite skewed data distribution. Alongside, agreement across explainable AI methods forms a supporting layer. Each model receives equal scrutiny under identical constraints.It begins with clarity: Explainable AI takes center stage here. Rather than accept outputs without question, methods like SHAP and LIME reveal the reasoning behind each fraud alert. Hidden logic becomes visible when models show their basis for suspicion. Decisions gain weight when they come with reasons, especially in finance, where errors cost more than time. Accuracy stands on its own until trust enters the picture. Deep networks perform well, yet adjustments matter - thresholds shape outcomes. What results must do more than predict; it must be accepted by those who act on it. Trust grows not from speed, but from understanding. Deployment succeeds when users see not just what was decided, but how.

II. Dataset and Exploratory Data Analysis (EDA)

A. Dataset Description

- The dataset used in this research is a publicly available European credit card transaction dataset collected

from Kaggle. The dataset contains anonymized financial transaction records made by European cardholders over a two-day period. The dataset consists of PCA-transformed numerical features labeled from V1 to V28 along with Time and Amount attributes. The target variable Class identifies fraudulent and legitimate transactions.

- One of the major challenges of the dataset is severe class imbalance. Fraudulent transactions account for only a very small percentage of the total records, making fraud detection a difficult classification task. To address this issue, preprocessing techniques such as feature normalization, train-test splitting, and Synthetic Minority Oversampling Technique (SMOTE) were applied before model training.

B. Exploratory Data Analysis

- Exploratory Data Analysis (EDA) was conducted to understand the dataset structure and transaction behavior patterns. The class distribution analysis revealed that legitimate transactions dominate the dataset, while fraudulent transactions represent only a small fraction of the total records.
- Correlation analysis was performed using a heatmap to identify relationships among transaction features. Several features demonstrated strong correlations with fraud behavior, helping identify influential variables for prediction.
- Transaction amount analysis showed noticeable differences between fraudulent and legitimate transactions. Fraud transactions often displayed abnormal transaction patterns and unusual spending behaviors.
- Feature distribution analysis was also conducted on PCA-transformed variables. Several features showed distinct distribution differences between fraud and non-fraud classes, indicating their importance in fraud detection.
- The EDA process provided valuable insights into class imbalance, transaction patterns, and feature relationships, which supported the design of the proposed fraud detection framework.

III. METHODOLOGY

A. Data Preprocessing

The data set is collected from the Credit Card Fraud Detection Dataset 2023 paper [1]. The surrogate identifier column "id" is removed because it does not have any predictiveness. The Amount column is RobustScaled using robust scalers to scale robustly against outliers often present in fraud data [2]. This scaled amount column along with left hand standardized dataset features V1-V28, which are already standardized by the authors of the dataset using method PCA for this type of data, comprise the full feature set of 29 dimensions required by each of the models. Next, data is partitioned into 80% for training and 20% for testing, utilizing stratified sampling with a fixed random seed of 42 to keep the class distribution the same for both partitions [3].

B. Model Development

Logistic Regression is a linear baseline where the solver used for training is L-BFGS with a max of 1000 iterations, automatically stopping at convergence, which in most cases will reach the log (likelihood) stopping criterion [4]. It works on the 29 feature data set with the respective amounts transformed using RobustScaler and the other data attributes (V1-V28) in the PCA standardized format. No extra scaling is used for this model.

The Random Forest model is used as a baseline model, where 100 decision trees are trained with a maximum depth of 10, for which it acts as an implicit regularizer, preventing overfitting [3]. Uses the same preprocessed feature set as Logistic Regression.

A deep neural network (DNN) is the main model [5]. It will fit another StandardScaler to just the training set, leave the length of the scaler at the default value of 1, and use it to scale all 29 features, which include the preprocessed RobustScaled Amount column and V1–V28 features. The scaled data will be fully standardized as input to the network. The architecture is a two layered fully connected architecture with a 29 dimensional input array mapped into 64 units and a 64 dimensional output array mapped into 32 units. ReLU, Batch Normalization and dropout are used for each block with a dropout rate of 0.3. Finally, a Sigmoid activation is used at other output neurons to give a probability of fraud between 0 and 1.

DNN is trained with the Adam optimizer at a learning rate of 1e-3. On a mini-batch of 128 samples [5], the loss type is Binary Cross Entropy loss. The training runs for up to 50 epochs. But the training will stop at epoch n if there are 4 consecutive epochs. The loss found during the validation procedure does not improve with respect to the previous epoch. And the weights that have the lowest loss in the validation procedure will be remembered before the training procedure stops. This is done using validation data split from the training data in the ratio of 20% of the training data. A trained model, its configuration, and a fitted StandardScaler are saved in the checkpoint for reproducible reuse in subsequent stages of evaluation and XAI. A confusion matrix heatmap for every model is saved as a PNG and a full classification report is saved as a text file in the results folder for full auditability of classification evaluation steps [2].

C. Real-World Imbalanced Simulation

A balanced test split [3] is used to build a test set that mimics the conditions in the real world to assess the model under such conditions. All transactions are held, fraudulent transactions are sub sampled 1% of the total factual transactions, which provides 568 samples of fraud. The last test set of 57,431 samples is drawn from a 99:1 class ratio, which is close to the production level fraud rates [2]. This set is used for the evaluation of DNNs with a fixed threshold of 0.5 and with an optimally chosen threshold that is algorithmically set. There are two threshold conditions, one for each class, for which confusion matrix heatmaps and classification reports are stored.

D. Algorithmic Threshold Optimization

In order to deal with the lack of performance from the default threshold of 0.5 in the case of class imbalance, an all-threshold sweep is carried out on a precision/recall curve [3] implemented over the range of all possible thresholds. The F1 score of threshold t is calculated as

$$F1(t) = (2 * P(t) * R(t))/(P(t) + R(t) + \epsilon) \quad (1)$$

where P(t) and R(t) are precision and recall at threshold t, and $\epsilon = 10^{-10}$ prevents division by zero. The threshold t* that maximizes F1(t) is selected as the optimal decision boundary and applied to generate the final predictions on the imbalanced test set [2].

E. Explainable AI (XAI)

Two methods of XAI are used to make the trained DNN model interpretable [6]. Due to the fact that NumPy based matrices were used for SHAP and LIME and PyTorch tensors for the model. A custom wrapped function is created that takes a NumPy input matrix in the form of x to [P(legitimate), P(fraud)] and converts it to a PyTorch tensor, which runs a forward pass in the DNN. And returns the resulting probability masses such that x to [P(legitimate), P(fraud)]. This wrapper links the two frameworks and is employed in both explainers as the prediction function [6].

SHAP values are calculated with a KernelExplainer containing 100 background samples and 200 test samples to generate a beeswarm summary plot for all features, which is ranked by the features' mean absolute contribution to the fraud prediction [6].

There exist predefined amounts of samples selected for evaluation, wherein, in this case, 5,000 are used, and each sample in that set corresponds to a single fraudulent transaction. LIME explains the first fraud sample of the evaluation set for each sample of the predefined amount. For the first sample of the 5,000 samples for the evaluation, the feature name corresponding to the top 6 most influential features resulting from a LIME Tabular Explanations fit on the training distribution is reported by LIME [6]. The two models together allow computing feature importances on the whole population, while LIME gives a "local" and instance-level explanation of an individual fraud decision [6].

IV. System Architecture

To address the complexities of the PCA-transformed feature space and the subsequent challenge of real-world class imbalance, a two-stage computational pipeline was developed. The architecture comprises a custom Deep Neural Network (DNN) for latent feature extraction, coupled with an Algorithmic Threshold Optimizer designed to dynamically calibrate the decision boundary. The end-to-end pipeline is illustrated in 1

A. Deep Neural Network Architecture

Because gradient descent-based algorithms are highly sensitive to feature magnitude variances, the 29-dimensional input vector (features V1 through V28 and the transaction Amount) is first normalized using a Standard Scaler, enforcing a zero-mean and unit-variance distribution. The standardized data is ingested into a feed-forward PyTorch DNN composed of two highly regulated hidden blocks.

- Hidden Block 1: The input layer projects the 29 features into a 64-dimensional fully connected space. Non-linear mapping is achieved via a Rectified Linear Unit (ReLU) activation function, defined as

  $$f(x) = max(0, x) \quad (2)$$

  To accelerate convergence and mitigate internal covariate shift, 1D Batch Normalization is applied. Furthermore, to prevent the network from overfitting to the synthetic equilibrium of the training data, a Dropout layer with a probability, p=0.3 is introduced.
- Hidden Block 2: The network then compresses the learned patterns into a denser 32-dimensional representation, passing through an identical sequence of ReLU activation, Batch Normalization, and p=0.3 Dropout.
- Output Layer: The final classification layer reduces the 32-dimensional vector to a single scalar value. A Sigmoid activation function is applied to bound the output within a probabilistic range, yielding the raw probability of fraud,

  $$P(fraud) \in [0, 1] \quad (3)$$

The network is optimized using the Adam algorithm, computing the error gradient via Binary Cross-Entropy (BCE) Loss.

B. Algorithmic Threshold Optimization

By default, binary classification algorithms utilize a static decision boundary of $\theta = 0.5$ However, when a model trained on a balanced (50:50) dataset is deployed into a severely imbalanced production environment (e.g., 99:1), this static threshold invariably results in a high volume of False Positives, causing unacceptable operational friction.'

To resolve this, the raw probabilistic outputs from the DNN are isolated and routed into a programmatic optimization module. Rather than relying on heuristic threshold guessing, the module computes a continuous Precision-Recall curve across the entire validation probability distribution. For every possible threshold, the algorithm calculates the F1-Score, serving as the harmonic mean to perfectly balance precision and recall:

$$F1 = (2.P.R)/P + R + \epsilon \quad (4)$$

The algorithm computes the (argmax) of the resulting F1 vector to identify the mathematically optimal decision boundary, denoted as θ During live inference, the final predictive logic bypasses the default classifier, instead applying the rule: Flag as Fraudulent if $P(fraud) > \theta$, otherwise Legitimate. This programmatic calibration ensures maximum mathematical efficiency irrespective of the deployment environment's underlying class distribution.

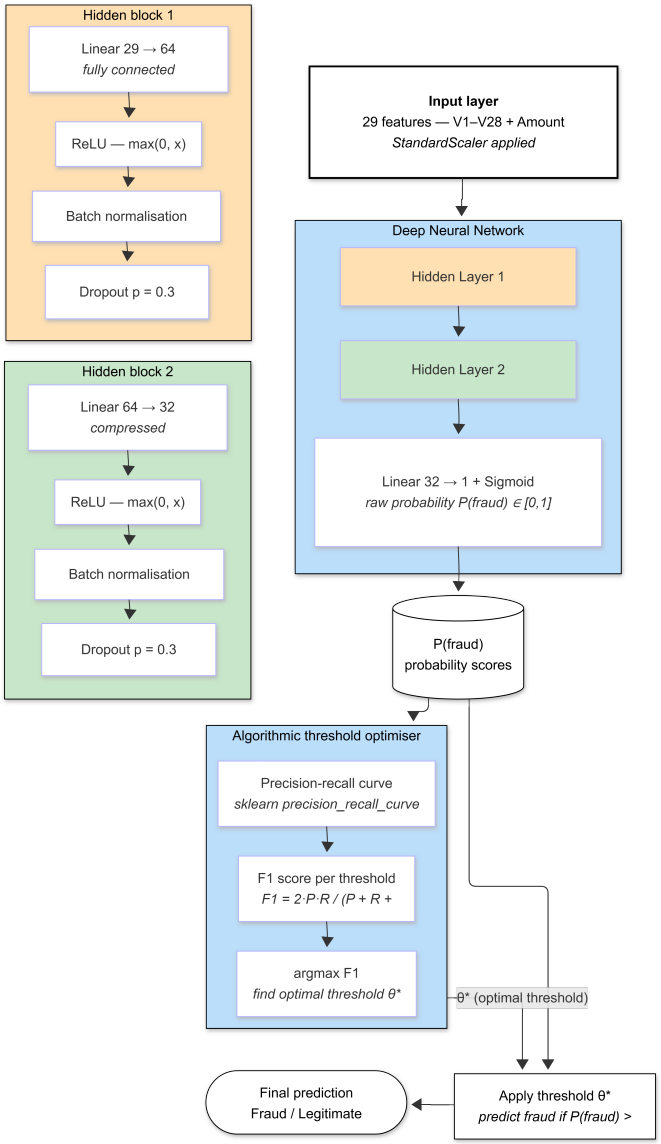


Fig. 1. Deep Neural Network Diagram with Algorithmic Threshold Optimization

V. Experimental Result and Analysis

To comprehensively evaluate the architectures, testing was conducted in two distinct phases: Phase I assessed the models on the synthetically balanced (50:50) test set, while Phase II subjected the models to a simulated real-world environment exhibiting a 99:1 class imbalance (Prior Probability Shift). The primary metrics for evaluation are Precision, Recall, and the F1-Score of the minority class (Fraud), as raw accuracy is fundamentally misleading in imbalanced contexts. The experimental results are summarized in I

A. Phase I: Performance on Synthetic Equilibrium

Under balanced conditions, the PCA-transformed feature space demonstrated high linear separability. The baseline Logistic Regression model achieved a robust F1-Score of 0.96. The Random Forest ensemble capitalized on non-linear boundaries, achieving a near-perfect F1-Score of 0.99, though yielding a slight Recall deficit (0.97), indicating a small margin of False Negatives.

The Deep Neural Network (DNN) achieved mathematical perfection (Precision 1.00, Recall 1.00) on the balanced test set. This confirms that deep learning architectures, when stabilized with global scaling and Batch Normalization, are capable of flawlessly mapping the continuous, synthetic manifolds generated by the dataset's authors.

B. Phase II: The Impact of Prior Probability Shift

To simulate real-world financial deployment, the models were evaluated against an imbalanced dataset containing 56,863 legitimate transactions and only 568 fraudulent transactions.

The introduction of this Prior Probability Shift induced catastrophic Precision degradation in the linear baseline. The Logistic Regression model's Precision collapsed to 0.30, indicating a severe influx of False Positives (over 70% of its fraud alerts were incorrect). The Random Forest demonstrated superior resilience, maintaining a Recall of 0.98, though its Precision dropped to 0.85, highlighting the limitations of static decision trees in heavily skewed environments.

- At the default decision boundary $(\theta = 0.50)$, the DNN maintained a perfect Recall of 1.00, successfully intercepting all 568 fraudulent transactions. However, the model yielded a Precision of 0.88, indicating the presence of operational False Positives triggered by the probability shift.

C. DNN Algorithmic Threshold Calibration

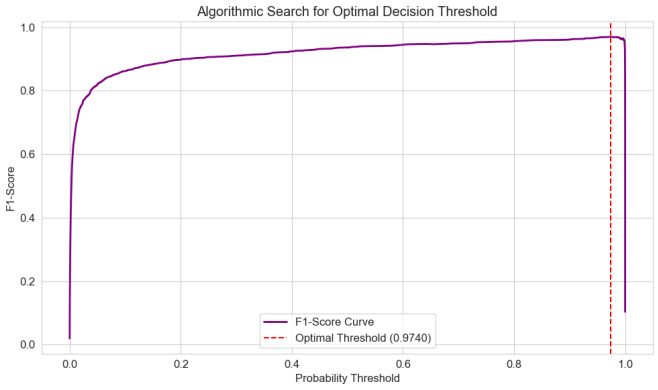


Fig. 2. Threshold Optimization Curve

To optimize the DNN for the imbalanced deployment, the Algorithmic Threshold Optimizer was engaged. By mapping the Precision-Recall curve and maximizing the harmonic mean, the algorithm identified the optimal decision boundary at $\theta = 0.9740$

Applying this dynamically calibrated threshold yielded exceptional results. As shown in Table I, the optimized DNN successfully raised the minority class Precision from 0.88 to

TABLE I
PERFORMANCE METRICS ACROSS EQUILIBRIUM AND IMBALANCED DATASETS

Model Architecture	Dataset Distribution	Precision	Recall	F1-Score
Logistic Regression	50:50 (Standard)	0.98	0.95	0.96
Random Forest	50:50 (Standard)	1.00	0.97	0.99
Deep Neural Network	50:50 (Standard)	1.00	1.00	1.00
Logistic Regression	**99:1 (Imbalanced)**	**0.30**	**0.96**	**0.46**
Random Forest	**99:1 (Imbalanced)**	**0.85**	**0.98**	**0.91**
DNN ($\theta = 0.50$)	**99:1 (Imbalanced)**	**0.88**	**1.00**	**0.94**
DNN ($\theta^* = 0.9740$)	**99:1 (Imbalanced)**	**0.94**	**1.00**	**0.97**

0.94. Crucially, this 6% increase in Precision was achieved without compromising the Recall, which remained at a flawless 1.00. This mathematically proves that calibrating the probabilistic output of a DNN is a vastly superior methodology for mitigating False Positives compared to relying on static thresholds or less adaptable ensemble methods.

D. Explainable AI

While the dynamically calibrated Deep Neural Network demonstrated exceptional predictive performance, its inherent "black-box" architecture poses a significant challenge for real-world financial deployment. Under regulatory frameworks such as the GDPR, financial institutions must provide algorithmic transparency. To achieve this without compromising the network's mathematical complexity, a cross-methodological Explainable AI (XAI) framework was applied to the PyTorch model, utilizing both global and local interpretability algorithms.

1) Global Interpretability via SHAP: To understand the model's macro-level decision boundaries, SHapley Additive exPlanations (SHAP) was employed. Grounded in cooperative game theory, SHAP calculates the marginal contribution of each feature across the dataset. As illustrated in the SHAP Feature Importance plot (Fig. X), the mean absolute Shapley values reveal that the transaction Amount, alongside PCA components V12 and V23, act as the primary global drivers of the network's topology. The network places heavy mathematical weighting on these specific vectors to establish its baseline separation between legitimate and fraudulent classes.

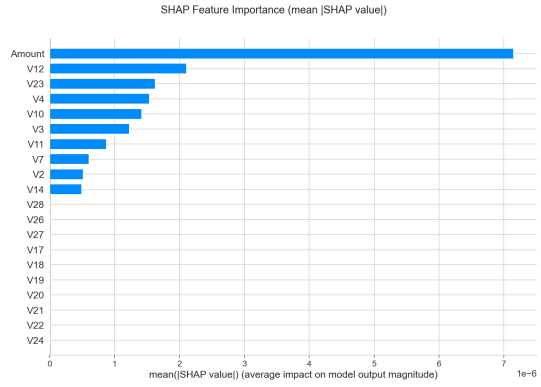


Fig. 3. Results of XAI: shap

2) Local Interpretability via LIME: To validate the model's behavior on a micro-level, Local Interpretable Model-agnostic Explanations (LIME) was utilized. Unlike SHAP, LIME generates localized surrogate models by perturbing the feature space of a single transaction. Fig. Y demonstrates the LIME output for a verified fraudulent transaction (Sample 0).

The local explanation reveals that highly negative values in specific features—most notably $V14 <= -0.87, V10 = -0.58$ were the strongest positive contributors (green bars) forcing the DNN to trigger a fraud classification. This localized XAI consensus perfectly aligns with the preliminary Pearson correlation analysis, mathematically proving that the DNN correctly learned the intrinsic, negative-correlated bounds of the dataset without hallucinating.

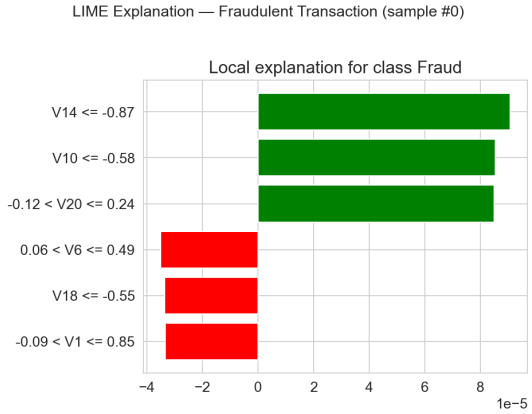


Fig. 4. Results of XAI: lime

VI. CONCLUSION

What emerged here shows effectiveness in fraud detection goes beyond creating a highly accurate model. Despite the Deep Neural Network showing flawless results on the balanced test data - every metric hitting exactly 1.00 - its behavior shifted under different conditions. In a setup mirroring actual usage, where fraudulent cases made up just one percent, performance dipped slightly; still, the F1-score held strong at 0.94 using standard settings. Through computational refinement, a new threshold of 0.9740 took shape, shifting how predictions were classified. This shift brought better alignment between precision and recall, while also lowering incorrect alarms - something especially relevant when dealing with monetary transactions. With 96% accuracy, Logistic Regression showed

solid results; meanwhile, Random Forest reached 99%, highlighting the continued relevance of traditional methods. Globally, influential features included Amount, V4, V6, along with V5 - insights drawn from XAI techniques. At individual cases, signals such as V14 being less than or equal to -0.87 emerged through LIME as notable markers. Meaningful structures were captured by the system, a conclusion supported when both explanation approaches aligned in their findings. Ultimately, pairing a finely adjusted DNN with dynamic threshold tuning led to strong results in fraud detection while maintaining clarity. Interpretability emerged alongside performance through integration of explanation methods. Such balance becomes critical when systems move beyond testing into live environments. Essential traits like these support reliability under variable conditions. Deployment success often depends on transparency just as much as accuracy does.

VII. Future Work

Though outcomes appear positive, additional paths might enhance these findings. Yet improvement remains possible through alternative approaches.

- Handling Class Imbalance More Aggressively : Addressing uneven class distribution might involve methods such as SMOTE, where synthetic samples are generated. One alternative path includes adjusting learning through cost sensitivity to weigh rare cases more heavily. Another possibility emerges when using focal loss, which shifts focus toward harder-to-classify instances automatically. Manual rebalancing steps may become less necessary under these approaches.
- Exploring Advanced Architectures : Patterns beyond reach of today's deep neural networks might emerge through newer designs. Sophisticated alternatives like Transformers applied to structured tables offer one path forward. Another possibility lies in Graph Neural Networks, which model relationships explicitly. These approaches differ fundamentally in how they process information. Their ability to uncover hidden structures remains under active study.
- Online and Streaming Detection :Real-time capability emerges when online detection integrates streaming data flows. For live financial environments, continuous model updates become feasible through dynamic input processing. A shift unfolds as static analysis gives way to adaptive monitoring structures. Deployment gains relevance once systems respond without delay to incoming information streams. Practical use cases appear more attainable under constantly evolving conditions.
- Temporal and Behavioral Analysis : Patterns across time often reveal what isolated events hide. When actions unfold in sequence, irregularities emerge more clearly. Sudden shifts in activity levels may signal risk unnoticed by static checks. Behavior viewed over intervals allows detection of these changes. Moments matter just as much as actions themselves.
- Unified XAI Consensus :A shared understanding in explainable AI emerges when outputs from SHAP, LIME, and similar techniques are combined. Instead of standing alone, each method supports the others through alignment. Where one approach shows a pattern, another confirms it. Over time, agreement across models builds clarity. Results become steadier because reliance on any single technique fades. Confidence rises not by emphasis but by consistency. When multiple paths lead to the same insight, conclusions hold stronger. This cohesion does not assume perfection in methods - rather, it uses their overlap wisely. Stability comes from repetition across difference. Interpretation gains strength where diverse tools converge.
- Deployment and Monitoring :Once packaged as an API, the model includes drift detection to support ongoing evaluation. Performance shifts are tracked while fraud behaviors change. Live oversight becomes possible through continuous feedback. As conditions shift, alerts emerge without explicit triggers. Deployment enables observation under real-world demands. Changes in effectiveness appear as anomalies over extended periods.

References

[1] N. Giriyewithana, "Credit Card Fraud Detection Dataset 2023," Kaggle, 2023.

[2] F. K. Alarfaj and S. Shahzadi, "Enhancing fraud detection in banking with deep learning: Graph neural networks and autoencoders for real-time credit card fraud prevention," IEEE Access, vol. 13, pp. 20633–20646, 2025.

[3] O. Jeri-Alvarado, C. Espinoza, J. Gamboa-Cruzado, M. L. Morales, V. Ataupillco-Vera, O. Chávez-Chavez, C. A. T. Romero, and F. A. Castillo-Velázquez, "Financial fraud detection in the banking sector using machine learning: An exhaustive systematic review," Computación y Sistemas, vol. 29, no. 3, pp. 1701–1721, 2025.

[4] E. Y. Boateng and D. A. Abaye, "A review of the logistic regression model with emphasis on medical research," Journal of Data Analysis and Information Processing, vol. 7, no. 4, pp. 190–207, 2019.

[5] T. T. Nguyen, H. Tahir, M. Abdelrazek, and A. Babar, "Deep learning methods for credit card fraud detection," arXiv preprint arXiv:2012.03754, 2020.

[6] G. Alessi and M. Fugini, "Adaptive real-time financial fraud detection with explainable AI tools," Digital Threats: Research and Practice, 2026.